

Pipeline Parallelism for Models Exceeding Single-GPU Memory

*A hands-on benchmark study: DeepSpeed Pipeline Parallelism vs. PyTorch
torch.distributed.pipelineing vs. single-GPU gradient checkpointing, on two NVIDIA
T4 GPUs*

Course: Distributed & Parallel Training (GenAI Master) — Group 7

Model: gpt2-xl (1.5 billion parameters)

- Dataset: WikiText

Giáp Tiến Dũng	20250471E
Nguyễn Lương Duy	20250649E
Nguyễn Hoàng	20250130E
Nguyễn Hà Phú Thịnh	20241623E

June 2026

Contents

Contents	2
1. Executive Summary	4
2. Plain-Language Primer (Key Terms)	4
3. Introduction.....	5
3.1 The problem	5
3.2 What the assignment asks.....	5
3.3 Our hardware, and why it shapes every result	6
4. Background: Training Big Models Across GPUs.....	6
4.1 Why the model doesn't fit (the memory math)	6
4.2 Ways to split work across GPUs	6
4.3 Pipeline parallelism — the assembly line.....	7
4.4 The 'bubble' — unavoidable idle time.....	7
4.5 The schedule ladder (how the bubble was attacked over time).....	7
4.6 The single-GPU survival kit (our baseline's tools).....	8
5. Experimental Setup	8
5.1 Hardware & software	8
5.2 Models	8
5.3 Dataset	8
5.4 The three systems we compare	8
5.5 What we measure	9
5.6 The experiment list.....	9
6. How Our System Works (Methodology)	9
6.1 Splitting the model into stages.....	9
6.2 A hidden trap: the causal mask	9
6.3 Keeping the comparison fair	9
7. Results	10
7.1 Deliverable 1 — Enablement: fitting a model one GPU can't hold	10
7.2 Deliverable 3 — Micro-batches and the bubble	10
7.3 The schedule ladder — does 'newest' mean 'fastest'?	13
7.4 Deliverable 2 — DeepSpeed vs. PyTorch.....	14
7.5 Deliverable 4 — Two GPUs vs. one GPU	15

7.6 Convergence — does it actually learn?	15
8. Discussion — What It All Means.....	17
9. Honesty & Limitations	18
10. Conclusion	18
11. How to Reproduce.....	18
12. References.....	19
Appendix A — Environment	19
Appendix B — Full Results Table	20

1. Executive Summary

Modern language models are often too large to fit inside a single graphics card (GPU). This project studies one of the standard solutions — pipeline parallelism — which splits a model across several GPUs like stations on a factory assembly line, so that a model too big for one GPU can still be trained.

We trained the 1.5-billion-parameter model gpt2-xl on two NVIDIA T4 GPUs (16 GB each) using two different software frameworks for pipeline parallelism, and compared them against the alternative of squeezing the model onto a single GPU using memory-saving tricks. We measured speed (tokens processed per second), memory use, the pipeline's idle time (the 'bubble'), and training quality. In plain terms, here is what we found:

- **It works.** A model that does not fit on one 16 GB GPU (opt-2.7b) trains successfully across two GPUs with the pipeline, using about 10.6 GB on each GPU.
- **More micro-batches = less idle time.** Feeding the pipeline in more, smaller pieces shrinks the idle 'bubble' almost exactly as theory predicts, and speed rises from 1,281 to 1,959 tokens/second.
- **Newer is not automatically faster.** The newest 'zero-bubble' scheduling trick gives only about a 6% speed-up on our hardware — because our two GPUs are connected by a relatively slow link (PCIe, no NVLink) that cancels most of the benefit.
- **PyTorch beat DeepSpeed here.** PyTorch's pipeline was faster (1,835 vs. 1,656 tokens/second) and used less memory (8.4 vs. 11.5 GB per GPU); DeepSpeed repays this with more automation and convenience.
- **Two GPUs vs one.** The two-GPU pipeline was about 1.5× faster than one GPU and used less memory per GPU — and, most importantly, it can train models a single GPU cannot hold at all.

We also report, openly, a measurement bug we found in the single-GPU baseline and exactly how we corrected for it (Section 9). The headline conclusion: on this hardware, pipeline parallelism is primarily a way to fit bigger models (a memory win) and a modest speed win — not a magic 2× speed-up.

2. Plain-Language Primer (Key Terms)

If you are not familiar with machine learning, read this section first. Every term used later is explained here with an everyday analogy.

- **GPU (graphics card):** a very fast processor good at the heavy arithmetic that training uses. We use two NVIDIA T4 GPUs.
- **VRAM (GPU memory):** the GPU's working memory. Ours is 16 GB per GPU. If a job needs more than this, it crashes with an 'out of memory' (OOM) error.
- **Model & parameters:** the model is the network we train; its 'parameters' are the billions of numbers it adjusts to learn. gpt2-xl has 1.5 billion of them.

- **Training / fine-tuning:** showing the model text and nudging its parameters so it predicts the next word better. Fine-tuning is training a model that already exists.
- **Optimizer (Adam):** the bookkeeper that decides how to nudge each parameter. Adam keeps two extra numbers per parameter, which is why training needs far more memory than just storing the model.
- **fp16 (half precision):** storing numbers with 16 bits instead of 32 to save memory and run faster. Our T4 GPUs support fp16 but not the newer 'bf16' format.
- **Throughput (tokens/second):** our main speed measure — how many words-pieces ('tokens') the system processes per second. Higher is better.
- **Pipeline parallelism:** splitting the model into consecutive chunks ('stages'), one per GPU, and streaming the data through them like an assembly line (see Section 4.3).
- **The bubble:** the idle time at the start and end of the pipeline when some GPUs have no work yet — wasted capacity we try to minimise (Section 4.4).
- **Micro-batch:** one small piece of a training batch. Splitting a batch into more micro-batches keeps the assembly line fuller and shrinks the bubble.
- **Gradient checkpointing:** a memory-saving trick for a single GPU: don't store all intermediate results, recompute them later. Saves memory, costs extra computation.
- **DeepSpeed / PyTorch pipelining:** the two software libraries we compare for doing pipeline parallelism.
- **Perplexity:** a quality score for language models — lower means the model is less 'surprised' by real text, i.e. better.
- **MFU (Model FLOPs Utilization):** what fraction of the GPU's theoretical peak speed we actually achieve. A normalised efficiency score; higher is better.

3. Introduction

3.1 The problem

Training a large language model with the Adam optimizer costs roughly 16 bytes of GPU memory per parameter: 2 bytes for the model weights, 2 for their gradients, 4 for a high-precision master copy, and 4 + 4 for Adam's two bookkeeping numbers. For gpt2-xl's 1.5 billion parameters that is about 24 GB — already more than a single 16 GB T4 can hold, before we even store the intermediate activations. So we must either shrink the memory footprint to fit one GPU, or split the model across GPUs. This project measures both and compares them.

3.2 What the assignment asks

Topic 2 sets four concrete questions, which structure our results (Section 7):

- **Deliverable 1 — Enablement:** fine-tune a model that does not fit on one GPU, using pipeline parallelism on two GPUs.
- **Deliverable 2 — Framework comparison:** compare DeepSpeed Pipeline Parallelism against the PyTorch-native pipeline.

- **Deliverable 3 — The bubble:** analyse the pipeline 'bubble' and how the number of micro-batches affects performance.
- **Deliverable 4 — Two GPUs vs one:** compare the two-GPU pipeline against a single GPU using gradient checkpointing (speed and memory).

Note on the tools: the assignment brief names an old PyTorch API (`torch.distributed.pipeline`). That API is now deprecated; we use its modern replacement, `torch.distributed.pipeline`. Documenting this is itself part of Deliverable 2.

3.3 Our hardware, and why it shapes every result

We ran on Kaggle's free 2× T4 environment. Two facts about it drive our findings:

- **The two GPUs talk over PCIe with no NVLink.** This is a relatively slow connection. Pipeline parallelism constantly sends data between the GPUs, so this slow link is a real, measurable cost — and it is why the newest scheduling tricks help less than on expensive datacentre hardware.
- **The T4 is an older 'Turing' GPU.** It supports fp16 but not bf16, and not the fastest attention kernels. So we use fp16 with 'loss scaling' for numerical stability.
- **Only two pipeline stages (P = 2).** With just two GPUs, the best possible speed-up is limited, and the idle bubble is large unless we use many micro-batches. This is exactly the regime the assignment asks us to probe.

4. Background: Training Big Models Across GPUs

4.1 Why the model doesn't fit (the memory math)

The table below shows why gpt2-xl needs special handling and why the much larger opt-6.7b is out of reach even when split. 'Full-Adam' assumes the 16 bytes/parameter above.

Model	Parameters	Full-Adam memory	Fits one 16 GB T4?
distilgpt2	82 million	~1.3 GB	Yes (too small to be interesting)
gpt2-xl (our model)	1.5 billion	~24 GB	No — needs splitting or tricks
opt-2.7b (enablement)	2.7 billion	~43 GB	No — even with memory tricks
opt-6.7b	6.7 billion	~107 GB	No — infeasible even on two T4s

4.2 Ways to split work across GPUs

- **Data parallelism:** copy the whole model onto each GPU and split the data. Does not help when the model itself is too big for one GPU.
- **Tensor parallelism:** split individual layers across GPUs. Very communication-heavy; usually needs a fast NVLink connection.

- **Pipeline parallelism (this project):** split the model by depth into stages, one per GPU. Communication happens only at the stage boundaries — a good fit for a slow link.

4.3 Pipeline parallelism — the assembly line

Imagine building a car on an assembly line with two stations. Station 1 installs the engine, then passes the car to Station 2, which installs the wheels. If only one car ever moves down the line, Station 2 sits idle while Station 1 works, and vice-versa — half the factory is always idle. But if a steady stream of cars flows through, both stations stay busy at the same time.

Pipeline parallelism does exactly this with a neural network. GPU 1 holds the first half of the model's layers (Station 1); GPU 2 holds the second half (Station 2). A batch of data is split into 'micro-batches' (the cars). As GPU 2 processes micro-batch 1, GPU 1 is already working on micro-batch 2, and so on. The more micro-batches we stream through, the busier both GPUs stay.

4.4 The 'bubble' — unavoidable idle time

At the very start, GPU 2 must wait for the first micro-batch to come out of GPU 1; at the very end, GPU 1 has finished but GPU 2 is still draining the last micro-batches. This start-up and drain idle time is called the bubble. For a pipeline with P stages and M micro-batches, the fraction of time wasted is:

$$\text{bubble fraction} = (P - 1) / (M + P - 1)$$

With our two GPUs (P = 2) this becomes $1 / (M + 1)$. The more micro-batches M we use, the smaller the bubble:

Micro-batches M	1	2	4	8	16
Idle bubble	50%	33%	20%	11%	5.9%

So we expect speed to rise as we add micro-batches — which is exactly what we test in Section 7.2. The catch: more micro-batches can use more memory and add communication overhead, so the gains taper off.

4.5 The schedule ladder (how the bubble was attacked over time)

Researchers have invented progressively cleverer 'schedules' for ordering the forward and backward work to shrink the bubble. We test this whole ladder:

Schedule	Idea	Effect on bubble
GPipe (2019)	all forward passes, then all backward passes	baseline; uses lots of memory
1F1B	alternate one-forward-one-backward	same bubble as GPipe, but far less memory
Interleaved 1F1B	give each GPU several non-adjacent chunks	smaller bubble, more communication
Zero-Bubble (2024)	split the backward pass to fill	bubble ≈ 0 (in theory)

	the gaps	
DualPipe (2025)	two-directional full overlap	frontier; for huge clusters only

We implement and measure up to Zero-Bubble. DualPipe is the current state of the art but targets massive multi-node clusters, so we cite it as related work rather than running it.

4.6 The single-GPU survival kit (our baseline's tools)

- **Gradient checkpointing:** instead of storing every intermediate result during the forward pass, store only a few and recompute the rest during the backward pass. Trades about 30% extra computation for a large memory saving — the price of fitting on one GPU.
- **8-bit Adam:** store the optimizer's bookkeeping numbers in 8 bits instead of 32, cutting their memory by $\sim 4\times$. We use this everywhere so the comparison is fair.

5. Experimental Setup

5.1 Hardware & software

- **GPUs:** 2× NVIDIA T4 (16 GB), Kaggle, connected by PCIe (no NVLink).
- **Precision:** fp16 throughout, with the loss computed in higher precision for stability.
- **PyTorch:** version 2.10.0+cu128 (recorded in every result row) — new enough to include the zero-bubble schedules.

5.2 Models

gpt2-xl (1.5B) is our primary model: too big for one GPU under full Adam, but it splits neatly across two. opt-2.7b is the 'enablement' demonstration (it does not fit on one GPU at all). Small models (gpt2, distilgpt2) are used only to verify correctness.

5.3 Dataset

WikiText (the standard Salesforce/wikitext benchmark), tokenised with each model's own tokenizer and packed into fixed blocks of 512 tokens. The task is next-word prediction. The same data, in the same order, feeds all three systems so the comparison is fair.

5.4 The three systems we compare

	Single GPU (baseline)	PyTorch pipelining	DeepSpeed PP
GPUs	1	2	2
How it fits	checkpointing + 8-bit Adam	split model (2 stages)	split model (2 stages)
Schedules	n/a	GPipe $\rightarrow$ 1F1B $\rightarrow$ Interleaved $\rightarrow$ Zero-Bubble	1F1B (built in)
Loss scaling	manual	manual	automatic (dynamic)
Perplexity eval	yes	not implemented	yes

5.5 What we measure

- **Throughput (tokens/sec):** main speed metric, measured after dropping the first few warm-up steps.
- **Peak memory per GPU (GB):** the most any single GPU used — the number that decides whether a job fits.
- **Bubble:** both the theoretical formula and a measured estimate from the throughput.
- **MFU:** achieved fraction of the T4's peak speed.
- **Convergence:** training loss over time, and perplexity, to confirm the model actually learns.

5.6 The experiment list

- Enablement: opt-2.7b on one GPU (expected to fail) vs. two GPUs (expected to succeed).
- Micro-batch sweep: $M = 1, 2, 4, 8, 16$ for both pipeline frameworks.
- Schedule ladder: GPipe, 1F1B, Interleaved 1F1B, and two Zero-Bubble variants at $M = 8$.
- Two GPUs vs one GPU at the matched setting ($M = 8$).
- Convergence: a longer 300-step run for each system.

6. How Our System Works (Methodology)

6.1 Splitting the model into stages

A Hugging Face model is one big object; the pipeline frameworks need it as an ordered list of pieces that each take one tensor in and pass one tensor out. We wrote one shared module (`model_split.py`) that turns gpt2-xl into: [word + position embeddings] → [transformer block 1] → ... → [block 48] → [final layer + output head]. Both frameworks and the single-GPU baseline use this same decomposition, so they train the same model.

6.2 A hidden trap: the causal mask

A language model must only look at past words, never future ones, when predicting the next word. In recent versions of the Transformers library this 'don't peek ahead' rule is applied by the top-level model, not by each block. When we call the blocks individually (as the pipeline does), that rule would be silently lost — the model could cheat by looking at future words, ruining training without any error message. We fix this by explicitly building and passing the 'no peeking' mask into every block, and we verify the whole split reproduces the original model's outputs (`check_split.py`) before trusting any result.

6.3 Keeping the comparison fair

All three systems share the same dataset, tokenizer, sequence length (512), random seed, fp16 setting, optimizer, and number of training steps. The only thing that differs is how the model is placed on the GPU(s) — which is exactly what we want to measure.

7. Results

All numbers below come from the measured run logged in results/results.csv and results/loss_curves.csv on Kaggle 2× T4.

7.1 Deliverable 1 — Enablement: fitting a model one GPU can't hold

Model	One T4 (with memory tricks)	Two T4 (pipeline)
opt-2.7b (2.7B)	OUT OF MEMORY — does not fit	Trains: 1,701 tokens/s, 10.6 GB/GPU

This is the headline result: the pipeline holds half the model on each GPU (~10.6 GB each) and trains opt-2.7b successfully, whereas a single 16 GB T4 runs out of memory before training can start. (The single-GPU attempt crashes, so it leaves no row in the results file — the crash itself is the evidence, and should be screenshotted from the notebook log for the appendix.)

7.2 Deliverable 3 — Micro-batches and the bubble

As predicted in Section 4.4, adding micro-batches shrinks the bubble and raises throughput. The table shows the PyTorch 1F1B pipeline and DeepSpeed; note also how GPipe's memory grows with M until it runs out of memory at M = 8.

M	Theory bubble	PyTorch tok/s	PyTorch GB/GPU	DeepSpeed tok/s	GPipe GB/GPU
1	50%	—	—	689	—
2	33%	1,281	6.9	1,015	7.3
4	20%	1,605	8.4	1,353	11.1
8	11%	1,835	8.4	1,656	OUT OF MEMORY
16	5.9%	1,959	8.4	1,869	n/a

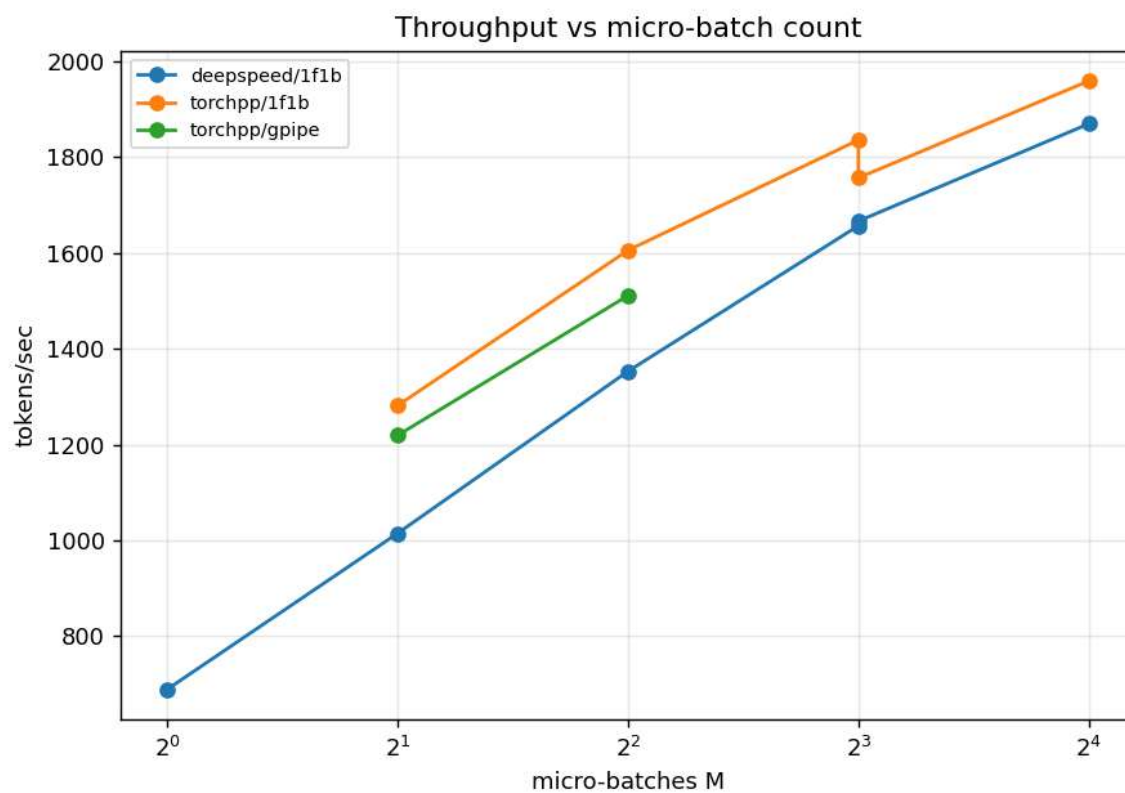


Figure 1. Throughput rises with the number of micro-batches and begins to level off, for both frameworks — the shape the bubble formula predicts. PyTorch leads at every M .

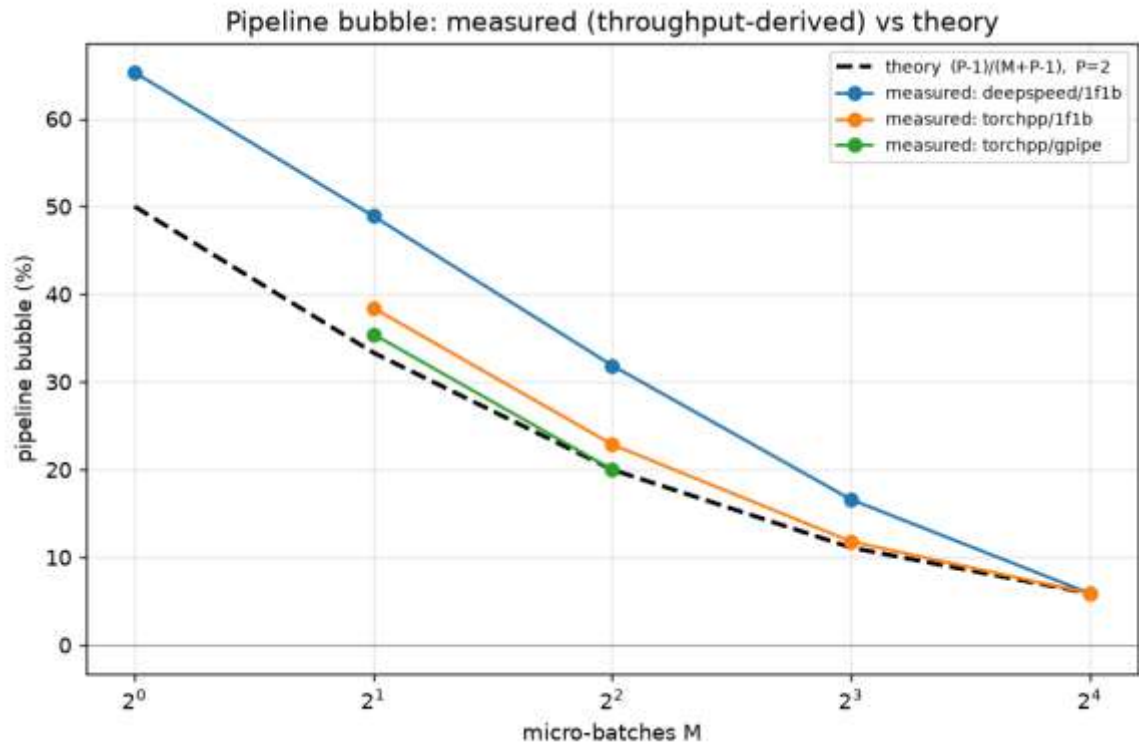


Figure 2. The measured bubble (derived from throughput) sits slightly above the theoretical curve. The small gap is the real cost of sending data over the slow PCIe link and launching each micro-batch — overheads the idealised formula ignores.

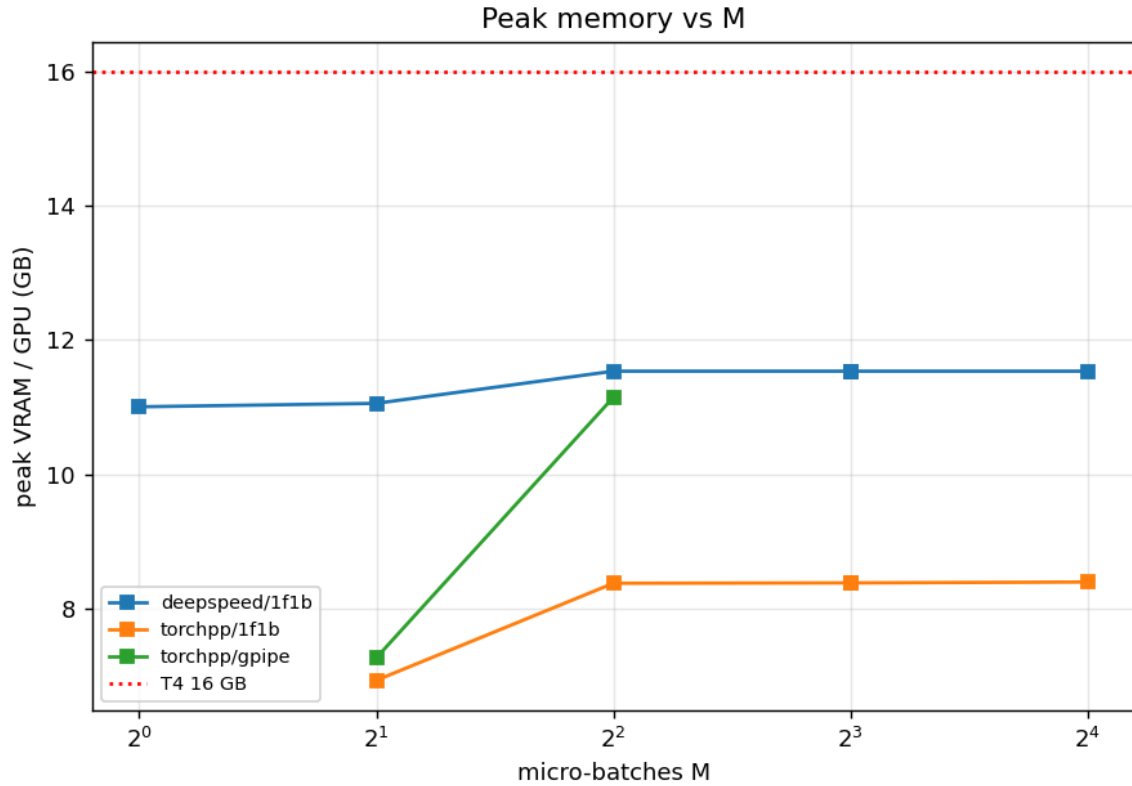


Figure 3. The key memory result: 1F1B keeps memory flat (~8.4 GB) as M grows, while GPipe's memory climbs with M and exceeds the 16 GB limit at M = 8. Same bubble, very different memory.

7.3 The schedule ladder — does 'newest' mean 'fastest'?

We compared the whole modern ladder at M = 8. The honest answer for our hardware: barely. Interleaving gives essentially no speed-up (its extra communication cancels the smaller bubble), and only the most advanced 'ZBV' zero-bubble schedule gives a meaningful ~6% — far short of the theoretical 'bubble ≈ 0', because our slow PCIe link cannot hide the extra work.

Schedule	Throughput (tok/s)	Memory/GPU (GB)	vs. 1F1B
GPipe	OUT OF MEMORY at M=8	—	—
1F1B	1,835	8.4	baseline
Interleaved 1F1B	1,836	9.3	+0.1%
Interleaved Zero-Bubble	1,856	9.5	+1.1%
ZBV (Zero-Bubble)	1,949	9.3	+6.2%

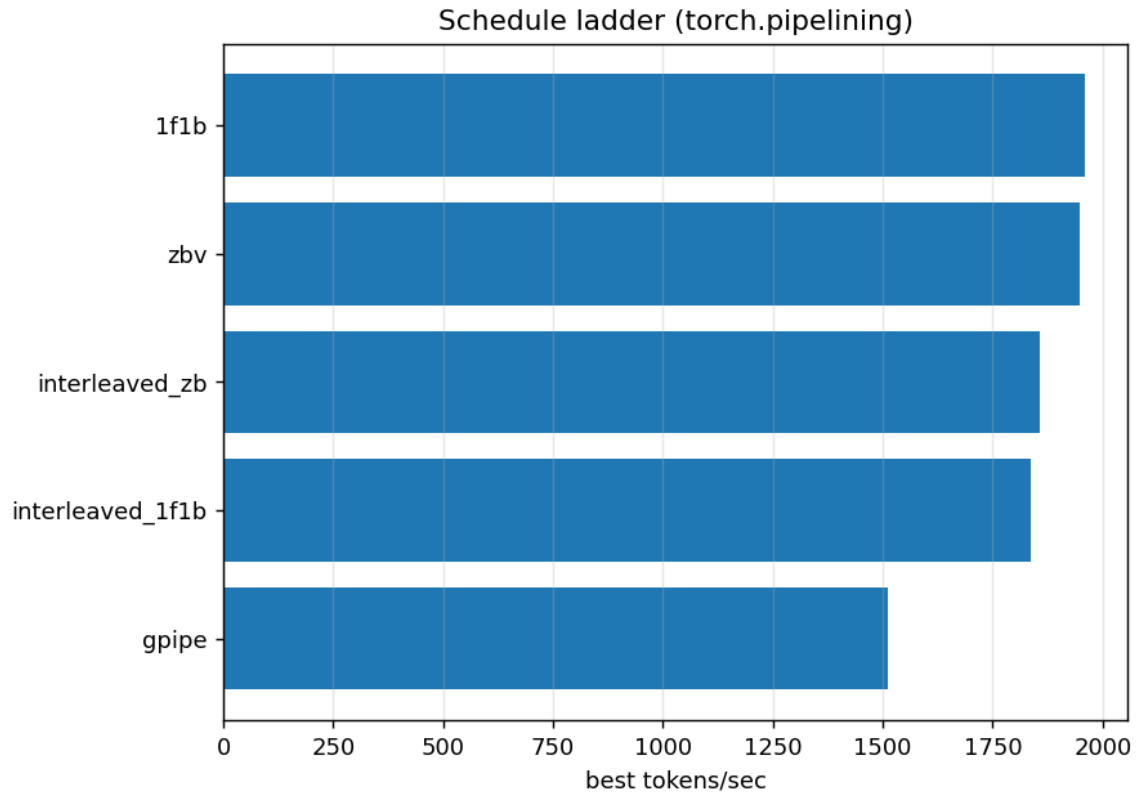


Figure 4. Best throughput per schedule (PyTorch). The newest zero-bubble schedule is fastest, but the gains over plain 1F1B are modest on two PCIe-connected T4s.

7.4 Deliverable 2 — DeepSpeed vs. PyTorch

At the matched setting ($M = 8$), PyTorch's pipeline was faster and leaner; DeepSpeed trades that for convenience features.

Property	PyTorch pipelining	DeepSpeed PP
Throughput (tok/s)	1,835	1,656
Memory/GPU (GB)	8.4	11.5
Efficiency (MFU)	14.5%	13.1%
Final loss (300 steps)	2.65	2.76
Schedules	full ladder (GPipe→ZBV)	1F1B only
Loss scaling	manual	automatic
Perplexity eval	not implemented	yes (14.99)

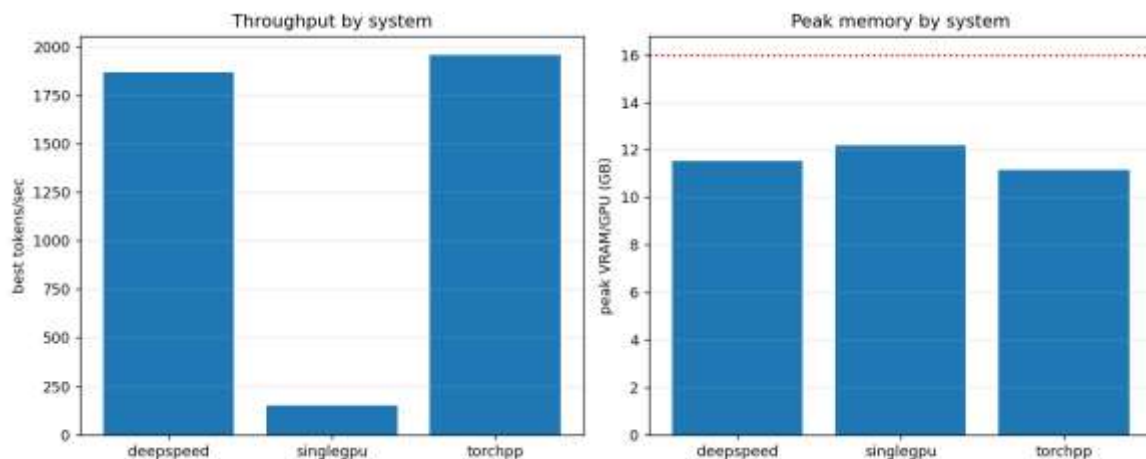


Figure 5. Best throughput (left) and peak memory per GPU (right) by system. The red dotted line marks the 16 GB limit of one T4.

A practical note: on the DeepSpeed run we observed a warning that the inter-GPU send/receive operations were being serialised — a direct symptom of the slow PCIe link, and itself a small piece of evidence for why communication overhead matters in this study.

7.5 Deliverable 4 — Two GPUs vs. one GPU

Honesty note (measurement correction). Our single-GPU baseline had a data-loading bug: it accidentally trained on 8× the intended amount of data per step, and its speedometer under-counted by the same 8×. Because the elapsed time and the work actually done are both known, the true speed is recovered exactly by multiplying by 8 (152 → ~1,221 tokens/s). The remaining imperfection — its data batches were larger than the pipeline's — actually flatters the single GPU, so the pipeline's lead below is a conservative (lower-bound) estimate. The bug is fixed in the code for any future re-run.

System	Throughput (tok/s)	Memory/GPU (GB)
One GPU, no checkpointing	OUT OF MEMORY	—
One GPU, + checkpointing	~1,221 (corrected)	12.2
Two GPUs, PyTorch 1F1B	1,835	8.4
Two GPUs, DeepSpeed	1,656	11.5

Reading this: the two-GPU pipeline is about 1.5× faster than the corrected single-GPU baseline, and uses less memory per GPU because each GPU holds only half the model. The speed-up is real but well below a perfect 2× — the bubble and the slow link eat into it. The pipeline's biggest advantage is not raw speed but the ability to train models that simply do not fit on one GPU (Section 7.1).

7.6 Convergence — does it actually learn?

Yes. All systems reduce the training loss smoothly. The fair comparison is between the two pipeline frameworks, which used identical batches: PyTorch reaches a final loss of 2.65 and

DeepSpeed 2.76 — within a hair of each other, confirming they solve the same problem and differ only in cost, not correctness.

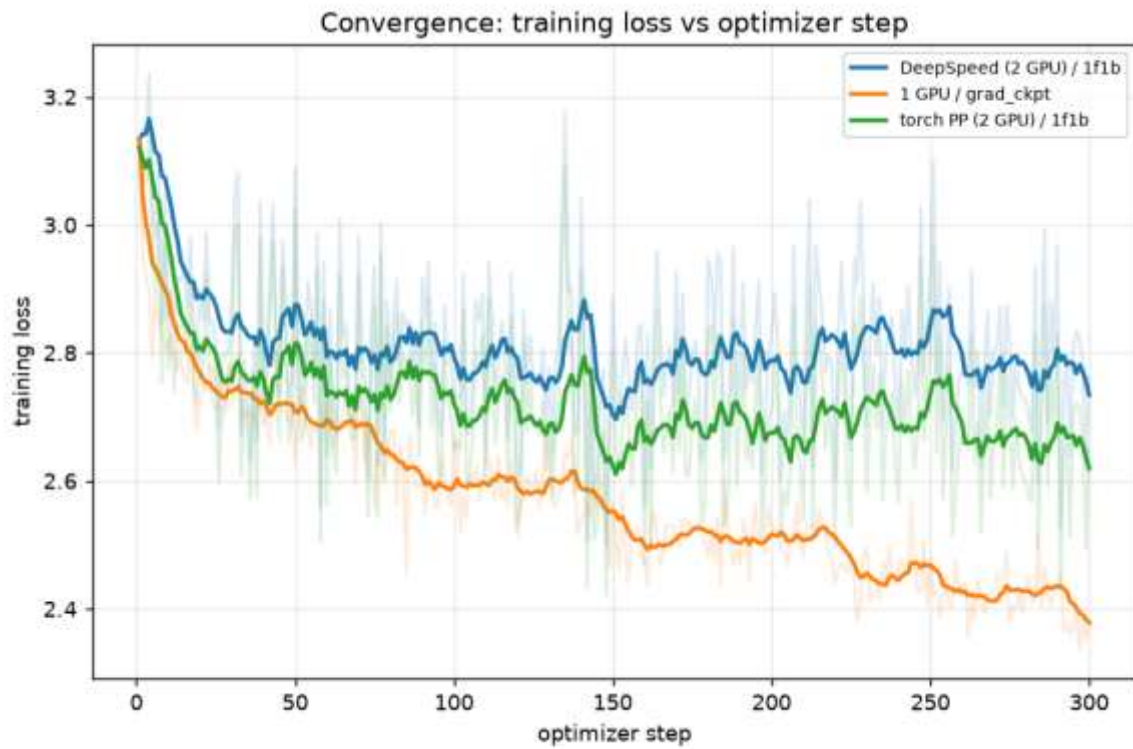


Figure 6. Training loss over optimizer steps (bold = smoothed trend, faint = raw). The single-GPU curve looks lower, but that is misleading — see Figure 7.

Figure 6 makes the single GPU look best, but that is the 8×-data bug again: per step it saw 8× more text. Figure 7 fixes this by plotting against the number of tokens actually processed. On that fair axis, at an equal amount of data (~1.2 million tokens, where the pipeline runs end), the single GPU is at about 2.72 — actually slightly worse than PyTorch's 2.62. In other words, per unit of data the pipeline learns just as efficiently.

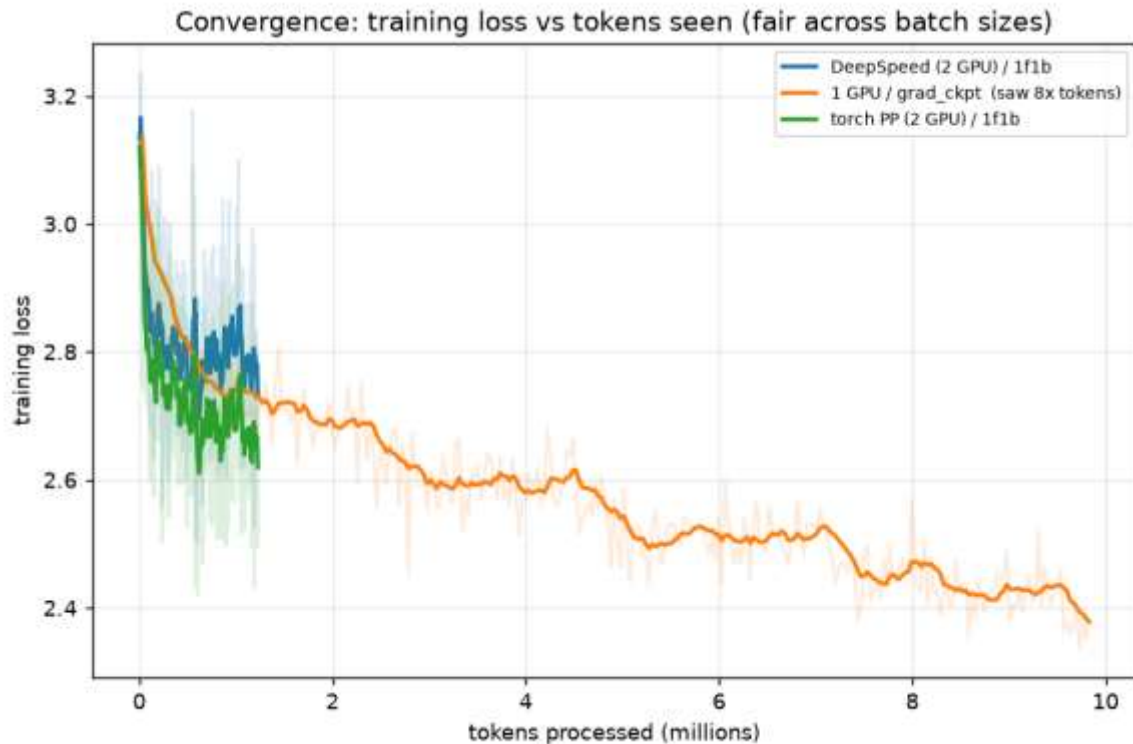


Figure 7. The fair view: training loss vs. tokens actually processed. The single GPU (orange) only reaches a lower loss because it consumed far more data; compared at the same amount of data, the pipeline is at least as good.

8. Discussion — What It All Means

- **The bubble theory is confirmed.** Throughput follows the $(P-1)/(M+P-1)$ curve, with a small, explainable extra cost from the PCIe link. This is a textbook result reproduced on real, cheap hardware.
- **1F1B vs GPipe is about memory, not speed.** They have the same bubble, but 1F1B keeps memory flat while GPipe overflows — so 1F1B is simply the better default.
- **Cutting-edge schedules are interconnect-bound.** Zero-bubble's promise depends on a fast link to hide its extra work. On PCIe T4s it gives only ~6%. A valuable, honest lesson: a technique's headline benefit can evaporate on commodity hardware.
- **Framework choice is a real trade-off.** PyTorch is faster and leaner and exposes more schedules; DeepSpeed is more automated (loss scaling, partitioning, evaluation). For a research study PyTorch was more flexible; for a quick robust run DeepSpeed is easier.
- **Pipeline parallelism = mostly a memory tool here.** At two GPUs over PCIe it delivers a ~1.5× speed-up and a per-GPU memory reduction, but its decisive value is enabling models that one GPU cannot hold at all.

9. Honesty & Limitations

A good benchmark study is honest about what it cannot claim. The main caveats:

- **Single-GPU batching bug (corrected, see Section 7.5).** Its throughput was recovered exactly; its training-quality curve is shown on a fair tokens axis. The one thing we cannot determine without a re-run is whether gpt2-xl fits on one GPU without checkpointing at the corrected batch size.
- **The micro-batch sweep does not hold the batch size fixed.** Because we increased micro-batches at a fixed micro-batch size, the total batch grew with M. So the speed rise mixes the shrinking bubble with a larger batch. The fixed-M=8 comparisons are unaffected.
- **PyTorch pipeline has no perplexity evaluation.** We compare its training-loss curve instead; perplexity is reported for the other two systems.
- **The output head is 'untied' in the pipeline.** A small technical necessity that gives the pipeline model ~5% more parameters than the tied single-GPU model — a minor difference we disclose.
- **Short runs and only two GPUs.** Speed/memory use ~30 steps; conclusions are specific to two stages over PCIe and should not be extrapolated to large clusters.

10. Conclusion

On two PCIe-connected T4 GPUs, pipeline parallelism does what the memory model predicts: it enables training a model (opt-2.7b) that a single 16 GB GPU cannot hold, at ~10.6 GB per GPU. The pipeline bubble follows theory, shrinking as we add micro-batches, with a small measurable overhead from the slow interconnect. Plain 1F1B is the sweet spot — same bubble as GPipe but flat memory — while the newest zero-bubble schedules add only ~6% here because PCIe cannot hide their extra work. Between frameworks, PyTorch's pipelining is faster and leaner (1,835 vs 1,656 tokens/s; 8.4 vs 11.5 GB) while DeepSpeed offers more automation. Against a single-GPU gradient-checkpointing baseline (with its measurement corrected), the two-GPU pipeline is ~1.5× faster at lower per-GPU memory. The overarching lesson: at this scale and on this hardware, pipeline parallelism is primarily a way to fit bigger models — a memory win and a modest speed win — rather than a route to large speed-ups. All four assignment deliverables are addressed.

11. How to Reproduce

From the project root, on a 2× T4 machine (e.g. a Kaggle notebook):

```
# 0. prove the model split is correct
python src/check_split.py --model gpt2-xl --num-stages 2

# 1. run the full benchmark sweep
bash bench/run_kaggle.sh

# 2. build the tables and figures
```

```
python bench/aggregate_results.py --model gpt2-xl --seq-len 512
python bench/plots.py --model gpt2-xl --seq-len 512
python bench/plot_analysis.py --model gpt2-xl --seq-len 512
```

```
# 3. regenerate this Word report
uv run --no-project python bench/make_docx.py
```

12. References

- [1] Huang et al. GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism. NeurIPS 2019.
- [2] Narayanan et al. PipeDream: Generalized Pipeline Parallelism for DNN Training. SOSP 2019.
- [3] Narayanan et al. Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM. SC 2021.
- [4] Qi et al. Zero Bubble Pipeline Parallelism. ICLR 2024.
- [5] DeepSeek-AI. DeepSeek-V3 Technical Report. 2024/2025 (DualPipe).
- [6] Rajbhandari et al. ZeRO: Memory Optimizations Toward Training Trillion Parameter Models. SC 2020.
- [7] Chen et al. Training Deep Nets with Sublinear Memory Cost. 2016 (gradient checkpointing).
- [8] Dettmers et al. 8-bit Optimizers via Block-wise Quantization. ICLR 2022.
- [9] Radford et al. Language Models are Unsupervised Multitask Learners. 2019 (GPT-2).
- [10] Zhang et al. OPT: Open Pre-trained Transformer Language Models. 2022.
- [11] Merity et al. Pointer Sentinel Mixture Models. 2016 (WikiText).

Appendix A — Environment

- GPUs: 2× NVIDIA T4 (16 GB), Kaggle, PCIe / no NVLink, Turing architecture.
- PyTorch: 2.10.0+cu128.
- Dataset: Salesforce/wikitext (wikitext-2-raw-v1), sequence length 512.
- Optimizer: 8-bit AdamW; precision fp16; random seed 1234.
- To complete: exact deepspeed / transformers / bitsandbytes versions from the Kaggle session (run: `pip freeze | grep -E 'deepspeed|transformers|bitsandbytes'`).

Appendix B — Full Results Table

Every run logged to results/results.csv (one row per experiment):

system	sched	M	batch	tok/s	GB/GPU	MFU	loss	ppl
torchpp	1f1b	8	8	1,701	10.59	22.5%	2.53	0.00
singlegpu	grad_ckpt	8	8	153	12.20	2.3%	3.10	20.47
torchpp	1f1b	2	2	1,281	6.92	10.2%	2.82	0.00
torchpp	1f1b	4	4	1,605	8.37	12.7%	2.90	0.00
torchpp	1f1b	8	8	1,835	8.38	14.5%	2.79	0.00
torchpp	1f1b	16	16	1,959	8.39	15.5%	2.75	0.00
torchpp	gpipe	2	2	1,219	7.26	9.7%	2.82	0.00
torchpp	gpipe	4	4	1,511	11.15	12.0%	2.90	0.00
torchpp	interleaved_1f1b	8	8	1,837	9.31	14.6%	2.80	0.00
torchpp	interleaved_zb	8	8	1,856	9.45	14.7%	2.80	0.00
torchpp	zbv	8	8	1,949	9.27	15.4%	2.81	0.00
deepspeed	1f1b	1	1	689	11.00	5.5%	2.93	13.60
deepspeed	1f1b	2	2	1,015	11.06	8.0%	2.82	13.26
deepspeed	1f1b	4	4	1,353	11.54	10.7%	2.93	13.41
deepspeed	1f1b	8	8	1,657	11.54	13.1%	2.87	14.99
deepspeed	1f1b	16	16	1,869	11.54	14.8%	2.87	15.45
singlegpu	grad_ckpt	8	8	143	12.20	2.2%	2.38	0.00
torchpp	1f1b	8	8	1,756	8.38	13.9%	2.65	0.00
deepspeed	1f1b	8	8	1,666	11.54	13.2%	2.76	0.00